05506026 Digital Image Processing (Java)

Lab Week 2

Objectives

1. Understand how to read and write bitmap image files in Java
2. Understand how to access width, height, and bit depth of the read image.
3. Create a class ImageManager to manipulate all image activities from now on
4. Understand how to access to each colour channel

Java BufferedImage

BufferedImage is a class in Java which you import the image data to a 2D buffer. It has API to retrieve the image's width, height, and many more information.

To use BufferedImage, package java.awt.image is required to be imported.

DIPLab.java

```
import java.awt.image.*;


public class DIPLab
{
  public static void main(String[] args)
  {
     BufferedImage img = null;
  }
}
```

File and ImageIO

To access to a file, we will use File class to identify the path first. Note that java.io is required to be imported.

DIPLab.java

```
import java.awt.image.*;
import java.io.*;


public class DIPLab
{
  public static void main(String[] args)
  {
     BufferedImage buffer = null;
     File f = null;


     f = new File("images/mandril.bmp");
  }
}
```

After the path of image file is identified, we can import the read file to BufferedImage using ImageIO class, which javax.imageio must be imported. Note that ImageIO must be surrounded by a try/catch block.

DIPLab.java

```
...
import javax.imageio.*;


public class DIPLab
{
  public static void main(String[] args)
  {
    ...
     f = new File("images/mandril.bmp");


    try
     {
        img = ImageIO.read(f);
     }
    Catch (IOException e)
     {
        System.out.println(e);
     }
  }
}
```

Image properties

Width, height, and bit depth can be obtained by methods of BufferedImage.

DIPLab.java

```
...
import javax.imageio.*;


public class DIPLab
{
  public static void main(String[] args)
  {
    ...


    int width = img.getWidth();
    int height = img.getHeight();
    int bitDepth = img.getColorModel().getPixelSize();
  }
}
```

Image writing

Now that if we want to copy the image file, we can write the BufferedImage to a new file using ImageIO again with another try/catch block.

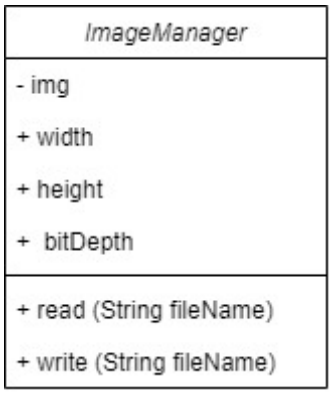
DIPLab.java

```
...
import javax.imageio.*;


public class DIPLab
{
  public static void main(String[] args)
  {
    ...


     f = new File("images/out.bmp");
    try
     {
      ImageIO.write(img,"bmp",f);
     }
    catch (IOException e)
     {
        System.out.print(e);
     }
  }
}
```

That concludes how to read and write image file using Java.

ImageManager Class

From the previous section, we can create a class which can be used to manipulate the image from now on. The class will store all the image data and has ability to read and write the file.



To implement this class, move the code in the previous section to each respective methods and the class will look like below.

ImageManager.java

```
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;
import java.io.IOException;

class ImageManager
{
   public int width;
   public int height;
   public int bitDepth;

   private BufferedImage img;
   private BufferedImage original;

   public ImageManager()
   {
   }

   public boolean read(String fileName)
   {
      try
      {
         img = ImageIO.read(new File(fileName));

         width = img.getWidth();
         height = img.getHeight();
         bitDepth = img.getColorModel().getPixelSize();
         System.out.println("Image "+fileName+" with "+width+" x "+height+" pixels ("+bitDepth+" bits per pixel) has been read!");
```

```
            return true;
        }
        catch (IOException e)
        {
            System.out.println(e);
            return false;
        }
    }

    public boolean write(String fileName)
    {
        try
        {
            ImageIO.write(img, "bmp", new File (fileName));
            System.out.println("Image "+fileName+" has been written!");

            return true;
        }
        catch (IOException e)
        {
            System.out.println(e);

            return false;
        }
        catch (NullPointerException e)
        {
            System.out.println(e);
            return false;
        }
    }
```

Colour channels

In Java, a pixel of image consists of several colour channels. A bitmap image has bit depth 24 which means it uses 3 bytes per pixel. Each byte will consist with one colour channel, and they align as Red, Green, and Blue.

The pixel in BufferedImage will have an integer value to represent RGB colours and this value can be obtained by using getRGB() method and pass the x and y position to its parameter.

The RGB colour received from this method can be separated into each colour channel using the following algorithm.

```
int color = img.getRGB(x, y);
int r = (color >> 16) & 0xff;
int g = (color >> 8) & 0xff;
int b = color & 0xff;
```

We can use this method to convert the image to only the RED channel presents.

ImageManager.java

```
class ImageManager
{
    ...
    public void convertToRed()
    {
        if (img == null)    return;

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = img.getRGB(x, y);
                int r = (color >> 16) & 0xff;
                int g = (color >> 8) & 0xff;
                int b = color & 0xff;

                color = (r << 16) | (0 << 8) | 0;

                img.setRGB(x, y, color);
            }
        }
    }
    ...
}
```

Also, we need another method to restore the image to its original.

ImageManager.java

```
class ImageManager
{
    ...
    private BufferedImage original;
    ...

    public boolean read(String fileName)
    {
        try
        {
            img = ImageIO.read(new File(fileName));

            ...

            original = new BufferedImage(width, height, img.getType());
            for (int y = 0; y < height; y++)
            {
                for (int x = 0; x < width; x++)
                {
                    original.setRGB(x, y, img.getRGB(x, y));
                }
            }
            ...
        }
        ...
    }

    ...

    public void restoreToOriginal()
    {
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                img.setRGB(x, y, original.getRGB(x, y));
            }
        }
    }
    ...
}
```

The program can be used with the main program as follows.

DIPLab.java

```
public class DIPLab
{
    public static void main (String[] args)
    {
        ImageManager im = new ImageManager();
        im.read("images/mandril.bmp");

        im.convertToRed();
        im.write("images/red.bmp");
    }
}
```

This concludes how to convert the image to only its red colour channel in Java.

Quest 001

Implement the program above to be able to copy an image “mandril.bmp”. The player who finishes this task will be awarded **15 EXP** and **1 MOVE**.

Deadline: 7 September 2025

Quest 002

Use the program above to implement two additional methods to convert the image to only its GREEN and BLUE colour channel.

The player who finishes this task will be awarded **10 EXP**.

Deadline: 7 September 2025

Quest 003

Use the program above to implement one additional method to convert the image to grayscale. One of the easiest ways is to take three-colour channels of each pixel and average their values.

The player who finishes this task will be awarded **20 EXP** and **1 MOVE**.

Deadline: 7 September 2025